

AI Support Agent for Uber_Support

Hiver SDE Intern Take-Home

Dataset: Customer Support on Twitter (Kaggle, [thoughtvector/customer-support-on-twitter](#))

Brand: Uber_Support, 56,307 brand replies in the full file

Golden set: 188 hand-labelled examples, [eval/golden_set.jsonl](#)

Stack: Qwen3-Embedding-0.6B retrieval, optional Qwen3-Reranker-0.6B, Gemini 2.0 Flash for classification, generation and judging

1 Problem framing

1.1 What “good” means for Uber

Uber support is not e-commerce support with different nouns. Three things shape what a good agent looks like here.

Safety outranks fluency, and it is not close. Around 2% of inbound messages reference a collision, harassment, a theft or police involvement. A fluent, polite, wrong reply to any of those is worse than no reply. So the system treats `safety_incident` as a hard stop: it is always escalated, and unlike every other intent it never gets a draft written at all. Putting a machine-written reply to an assault report in front of a human to approve is still the machine composing that reply, and that is not a judgment the system should be making on its own.

Twitter is a triage funnel, not a resolution channel. Reading Uber’s own historical replies makes this obvious. The overwhelming majority redirect to DM or the in-app help centre, because fares, trip receipts and driver identity cannot be discussed on a public timeline. So “good” is routing correctly and setting expectations, not resolving the issue in 280 characters. The system is not penalised for failing to solve anything publicly, because Uber’s own agents do not either.

No inventing policy. The generator is grounded in retrieved past resolutions specifically so it does not improvise refund amounts, ETAs or compensation. A plausible sounding invented promise is a support liability, not a good reply.

1.2 What was deliberately not built

- **Multi-turn context.** Only the first brand reply in each thread is used. Uber threads redirect to DM almost immediately, so later turns are logistics rather than resolution. This has a real cost, visible in Failure Mode 4.
- **Non-English handling.** A character-script filter drops non-Latin-script messages, mostly Japanese and Hindi. Building multilingual support would have meant either a translation layer or a multilingual judge, and neither would have been evaluated properly in the time available. Spanish and other Latin-script languages slip through the filter, which is a known gap rather than a solved problem.
- **Fine-tuning.** Uber’s historical agents are inconsistent. Fine-tuning on their replies would encode that inconsistency as ground truth. Few-shot prompting plus retrieval keeps the historical data as evidence rather than as a target.
- **Banking77.** Offered as an optional secondary source for intent work. Its 77 banking intents do not map onto rideshare traffic, and the brief asks for intents defined from the data, so borrowing an unrelated taxonomy would have worked against the point.

2 Results against two baselines

All numbers are computed over the 188-example golden set by `eval/run_eval.py`.

2.1 Intent classification

System	Accuracy	Macro F1	What it is
Baseline 1: keyword match	24.5%	0.236	first substring match wins
Baseline 2: TF-IDF + LogReg	35.1%	0.206	70/30 split, unigrams and bigrams
This system: few-shot LLM	36.2%	0.316	structured JSON, safety pre-filter

The keyword baseline is deliberately weak and it shows: it cannot place 131 of 188 messages at all and dumps them into `other`. It also cannot tell “the app crashed” from “my driver crashed”, which for a rideshare brand is the difference between a bug report and a collision.

The TF-IDF baseline is the more honest bar. It wins on raw accuracy by learning the head classes and ignoring the tail, which is exactly what its low macro-F1 (0.206) exposes. Accuracy alone would flatter it.

2.2 Escalation and reply quality

Metric	Result	Reading
Escalation accuracy	59.6%	agreement with human escalate/hold decision
Escalation Cohen’s κ	0.191	the number that actually matters, see 4
Zero-draft safety routing	9	messages that bypassed generation entirely
Reply quality, judge mean	2.99	1 to 5 across four rubric dimensions
Judge parse failures	0	malformed JSON from the judge

2.3 Is the LLM judge trustworthy

An LLM judge is worth nothing unless it tracks a human. 30 replies were scored independently by hand and compared against the judge.

Agreement metric	Value	Reading
Exact match	66.7%	identical score on 20 of 30
Within 1 point	100.0%	no catastrophic disagreement
Cohen’s κ	0.513	moderate agreement
Judge mean vs human mean	3.27 vs 3.00	the judge runs lenient

The gap between those two means is the useful finding. The judge is systematically kinder than a human, and consistently so on vague, non-committal replies. Asked to rate “We’ve followed up via DM!” against a customer who wanted a specific answer, the judge gave 3 and the human gave 2, noting it was evasive. Every reply quality number in this report should be read with that 0.27 point optimism bias in mind.

3 Failure analysis

Five failure modes, with real examples pulled from the evaluation artifacts. Pair IDs trace back to `eval/golden_set.jsonl`.

3.1 Safety incidents classified as ordinary complaints

This is the worst failure the system has, so it goes first.

Recall on `safety_incident` was 6 of 11. Five genuine safety reports were classified as something else, and two of those were not escalated at all:

- 1662425_1662424: “i’ve had multiple male drivers record me. is this allowed? they’re all using the same type of camera. very uncomfortable”. Classified as negative feedback. Not escalated.
- 2508288_2508287: a rider describing being publicly berated by a driver over their vision and cognitive ability. Classified as negative feedback. Not escalated.
- 1789981_1789980: “my uber driver stole my phone and uber won’t help me”. Classified as a complaint. Escalated, but without the zero-draft guarantee.

Hypothesis. The hard keyword pre-filter catches explicit emergency vocabulary (“police”, “assault”, “911”) and the few-shot examples show physical injury. None of those cover harassment described calmly, covert recording, or theft. The classifier reads measured language as measured severity. The lesson is that the safety category is defined by the nature of the event, not the temperature of the writing, and the prompt does not currently say so.

3.2 Hallucinated names copied out of retrieved context

330198_330197. Customer message in full: “Dear plz help....”. The system drafted: “I’m here to assist you, Bobby. Please provide more details about what assistance is needed.”

There is no Bobby. The message carries almost no content, so retrieval returned a weak nearest neighbour in which a human agent had addressed someone by that name, and the generator copied it. **Hypothesis:** with nothing to ground against, the model treats retrieved text as a template to fill rather than as evidence. Low retrieval similarity should suppress generation, not proceed with it.

3.3 Replies that reference links the cleaning step removed

960484_960482. Customer: “hey, so we took a thermus from our driver by mistake and we don’t know how to return it. Help!!”. Draft reply: “Hi there! Please reach out to and fill out the 4 boxes at the bottom so we can get in touch.”

“reach out to” with nothing after it. **Hypothesis:** `data_prep.py` strips URLs from both sides of every pair. Retrieved examples therefore contain the grammar of linking without the link, and the generator reproduces the dangling phrasing. The cleaning step and the generator disagree about what the data means.

3.4 Single-word follow-ups with no context

1279123_1279122. The entire customer message is “Sent”.

Labelled `other` by hand because in isolation it genuinely is unclassifiable. **Hypothesis:** this is the direct cost of the first-reply-only decision in 1.2. “Sent” means something specific if the previous turn was “please DM us your trip ID”, and nothing at all without it. The system is being asked a question that cannot be answered from the input it is given, and the golden set label reflects that honestly rather than pretending a correct answer exists.

3.5 Safety false positives on ordinary frustration

The same pre-filter that misses calm harassment reports also over-fires. Six messages were pushed into `safety_incident` that were not: lost keys in a car (`order_delivery`), a Pool driver who did not arrive but charged anyway (`billing_refund`), an account compromise (`account_access`).

Hypothesis: substring matching on words like “police” and “accident” has no notion of who the sentence is about. These errors are much cheaper than 3.1, since the cost is an unnecessary human review rather than a missed incident, and that asymmetry is deliberate. It is still a precision problem worth naming.

4 What is misleading about my headline number

4.1 The evaluation had a retrieval leak, and the first set of numbers was invalid

Worth stating plainly because it is the largest thing that went wrong. Golden-set messages also live in the retrieval index. In the first evaluation run, 176 of 188 predictions retrieved the query’s own pair as the top result at similarity exactly 1.0, which means the generator was shown the ground-truth reply before writing its own.

Two consequences. Reply quality was inflated, because the model was paraphrasing an answer it had been handed. More subtly, the grounding-similarity branch of the escalation rule never fired once, because similarity never dropped below the 0.55 threshold. One third of the escalation policy was dead code and the numbers gave no hint of it.

`RetrievalIndex.query` now takes `exclude_pair_id` and the pipeline passes each row’s own ID. The affected results are archived unused in `eval/stale_pre_fix/`. The general lesson is that a similarity of exactly 1.0 appearing in 94% of rows should have been read as a bug immediately rather than as strong retrieval.

4.2 Escalation accuracy is the wrong number to quote

The golden set splits 94 to 94 on escalate versus hold. On a perfectly balanced set a coin flip scores 50% and a system that escalates everything scores 50%, so raw accuracy has a floor that looks respectable while meaning nothing. Cohen’s κ is reported alongside it for that reason, and κ is the number to argue about.

Worth noting that the balance is an artefact of stratified sampling. Real Uber traffic is nowhere near 50% escalation-worthy, so neither figure transfers to production volumes without recalibration.

4.3 A taxonomy change silently broke the intent numbers

Partway through, an extra intent (`feedback_negative`, splitting venting from actionable complaints) was added to the classifier. The golden set had already been labelled without it. The system then predicted a label the ground truth could not contain 55 times, which accounted for 44 of 138 errors and pushed accuracy from roughly 0.40 down to 0.27.

Nothing in the evaluation flagged this. It looked like a bad model. The split is a real distinction and worth revisiting, but only together with a relabelling pass, so the intent was removed to match what the golden set actually encodes.

4.4 The TF-IDF baseline is trained on 131 examples

Ten classes, 131 training rows, roughly 13 per class, several classes in single digits. It is a directional sanity check and nothing more. It should not be read as a ceiling on what classical methods could do here.

4.5 $N = 188$ is small, and the interval is wide

A 95% Wilson interval on a proportion near 0.40 at $N = 188$ spans roughly ± 7 points. Differences smaller than that between systems are noise. `service_outage` drew zero labels in the sample, so any per-class metric for it is computed over an empty support set and should be ignored rather than interpreted.

4.6 Nothing here measures faithfulness

The judge rates whether a reply reads as correct. It does not check the reply against the retrieved context. A reply that contradicts the historical resolution it was grounded on can still score well by sounding right, and the “Boby” failure in 3.2 is exactly that category of error slipping past a quality score.

5 What one more week would go on

1. **Fix the safety recall problem first.** 6 of 11 is not shippable. Concretely: add few-shot examples covering harassment, covert recording and theft described in calm language, and evaluate safety recall as a standalone metric with its own threshold rather than letting it sit inside macro-F1 where 11 examples cannot move the number.
2. **Make low retrieval similarity suppress generation.** The threshold currently only influences escalation. When the best match is weak, the right behaviour is to produce no draft rather than a confident one built from an unrelated example. That fixes 3.2 directly.
3. **Two turns of thread context.** Enough to resolve “Sent” and “DM done”. Full dialogue state tracking is not needed for the gain here.
4. **Tune the escalation thresholds instead of guessing them.** 0.75 and 0.55 were never validated. With a balanced golden set available, a sweep with per-intent thresholds that weight safety recall above overall accuracy is straightforward.
5. **Relabel for the venting versus demand split.** It is a genuine distinction that maps to different handling. It needs the golden set relabelled first, in that order, which is the lesson from 4.3.

Reproduction: `README.md`. Decisions and their reasoning: `decision_log.md`. Golden set sampling and labelling method: `eval/golden_set_schema.md`. All figures regenerate via `python scripts/report_numbers.py`.